

CSE 3204
Formal Language, Automata and Computability



Lecture 15
Equivalence of PDA and CFG

From Grammar to PDA

- Given a CFG G , construct a PDA simulating the leftmost derivations of G .
- Any left sentential form not a terminal string, can be written as $xA\alpha$ where A = the leftmost variable, x = terminals appearing to the left of A and α = the string of terminals and variables appearing to the right of A .
- $A\alpha$ = the tail of the left sentential form. If a left sentential form consists of terminals only then its tail is ϵ
- Construction of a PDA from a grammar is to have the PDA simulate the sequence of left sentential forms that the grammar uses to generate a given terminal string w .
- The tail of each sentential form $xA\alpha$ appears on the stack with A at the top. x is the consumed input prefix. If $w=xy$, then y will be the remaining input.
- Suppose the PDA is in an ID $(q, y, A\alpha)$ representing a left sentential form $xA\alpha$. It guesses the production to use to expand $A \rightarrow \beta$. Remove A from the stack and place β on the top. Now the ID of the PDA is $(q, y, \beta\alpha)$.

From Grammar to PDA

Now $(q, y, \beta\alpha)$ may not be a representation of the next left-sentential form, because β may have a prefix of terminals. In fact, β may have no variables at all, and α may have a prefix of terminals. Whatever terminals appear at the beginning of $\beta\alpha$ need to be removed, to expose the next variable at the top of the stack. These terminals are compared against the next input symbols, to make sure our guesses at the leftmost derivation of input string w are correct; if not, this branch of the PDA dies.

If we succeed in this way to guess a leftmost derivation of w , then we shall eventually reach the left-sentential form w . At that point, all the symbols on the stack have either been expanded (if they are variables) or matched against the input (if they are terminals). The stack is empty, and we accept by empty stack.

From Grammar to PDA

The above informal construction can be made precise as follows. Let $G = (V, T, Q, S)$ be a CFG. Construct the PDA P that accepts $L(G)$ by empty stack as follows:

$$P = (\{q\}, T, V \cup T, \delta, q, S)$$

where transition function δ is defined by:

1. For each variable A ,

$$\delta(q, \epsilon, A) = \{(q, \beta) \mid A \rightarrow \beta \text{ is a production of } G\}$$

2. For each terminal a , $\delta(q, a, a) = \{(q, \epsilon)\}$.

Example 6.12: Let us convert the expression grammar of Fig. 5.2 to a PDA. Recall this grammar is:

$$\begin{aligned} I &\rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\ E &\rightarrow I \mid E * E \mid E + E \mid (E) \end{aligned}$$

The set of input symbols for the PDA is $\{a, b, 0, 1, (,), +, *\}$. These eight symbols and the symbols I and E form the stack alphabet. The transition function for the PDA is:

$$\text{a) } \delta(q, \epsilon, I) = \{(q, a), (q, b), (q, Ia), (q, Ib), (q, I0), (q, I1)\}.$$

$$\text{b) } \delta(q, \epsilon, E) = \{(q, I), (q, E + E), (q, E * E), (q, (E))\}.$$

$$\begin{aligned} \text{c) } \delta(q, a, a) &= \{(q, \epsilon)\}; \delta(q, b, b) = \{(q, \epsilon)\}; \delta(q, 0, 0) = \{(q, \epsilon)\}; \delta(q, 1, 1) = \\ &\{(q, \epsilon)\}; \delta(q, (, () = \{(q, \epsilon)\}; \delta(q,),)) = \{(q, \epsilon)\}; \delta(q, +, +) = \{(q, \epsilon)\}; \\ &\delta(q, *, *) = \{(q, \epsilon)\}. \end{aligned}$$

Note that (a) and (b) come from rule (1), while the eight transitions of (c) come from rule (2). Also, δ is empty except as defined by (a) through (c). $\square$

Theorem 6.13: If PDA P is constructed from CFG G by the construction above, then $N(P) = L(G)$.

PROOF: We shall prove that w is in $N(P)$ if and only if w is in $L(G)$.

(If) Suppose w is in $L(G)$. Then w has a leftmost derivation

$$S = \gamma_1 \xRightarrow{lm} \gamma_2 \xRightarrow{lm} \cdots \xRightarrow{lm} \gamma_n = w$$

We show by induction on i that $(q, w, S) \xRightarrow{P}^* (q, y_i, \alpha_i)$, where y_i and α_i are a representation of the left-sentential form γ_i . That is, let α_i be the tail of γ_i , and let $\gamma_i = x_i \alpha_i$. Then y_i is that string such that $x_i y_i = w$; i.e., it is what remains when x_i is removed from the input.

BASIS: For $i = 1$, $\gamma_1 = S$. Thus, $x_1 = \epsilon$, and $y_1 = w$. Since $(q, w, S) \xRightarrow{P}^* (q, w, S)$ by 0 moves, the basis is proved.

INDUCTION: Now we consider the case of the second and subsequent left-sentential forms. We assume

$$(q, w, S) \xRightarrow{P}^* (q, y_i, \alpha_i)$$

and prove $(q, w, S) \xRightarrow{P}^* (q, y_{i+1}, \alpha_{i+1})$. Since α_i is a tail, it begins with a variable A . Moreover, the step of the derivation $\gamma_i \Rightarrow \gamma_{i+1}$ involves replacing A by one of its production bodies, say β . Rule (1) of the construction of P lets us replace A at the top of the stack by β , and rule (2) then allows us to match any terminals on top of the stack with the next input symbols. As a result, we reach the ID $(q, y_{i+1}, \alpha_{i+1})$, which represents the next left-sentential form γ_{i+1} .

To complete the proof, we note that $\alpha_n = \epsilon$, since the tail of γ_n (which is w) is empty. Thus, $(q, w, S) \xRightarrow{P}^* (q, \epsilon, \epsilon)$, which proves that P accepts w by empty stack.

(Only-if)

- If $(q, x, A) \vdash_P^* (q, \epsilon, \epsilon)$, then $A \xRightarrow[G]^* x$.

The proof is an induction on the number of moves taken by P .

BASIS: One move. The only possibility is that $A \rightarrow \epsilon$ is a production of G , and this production is used in a rule of type (1) by the PDA P . In this case, $x = \epsilon$, and we know that $A \Rightarrow \epsilon$.

INDUCTION: Suppose P takes n moves, where $n > 1$. The first move must be of type (1), where A is replaced by one of its production bodies on the top of the stack. The reason is that a rule of type (2) can only be used when there is a terminal on top of the stack. Suppose the production used is $A \rightarrow Y_1 Y_2 \cdots Y_k$, where each Y_i is either a terminal or variable.

The next $n - 1$ moves of P must consume x from the input and have the net effect of popping each of Y_1, Y_2 , and so on from the stack, one at a time. We can break x into $x_1 x_2 \cdots x_k$, where x_1 is the portion of the input consumed until Y_1 is popped off the stack (i.e., the stack first is as short as $k - 1$ symbols). Then x_2 is the next portion of the input that is consumed while popping Y_2 off the stack, and so on.

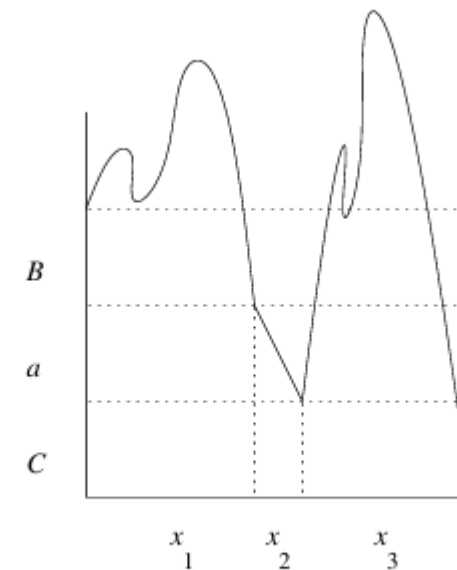


Figure 6.9: The PDA P consumes x and pops BaC from its stack

Formally, we can conclude that $(q, x_i x_{i+1} \cdots x_k, Y_i) \vdash^* (q, x_{i+1} \cdots x_k, \epsilon)$ for all $i = 1, 2, \dots, k$. Moreover, none of these sequences can be more than $n - 1$ moves, so the inductive hypothesis applies if Y_i is a variable. That is, we may conclude $Y_i \xRightarrow{*} x_i$.

If Y_i is a terminal, then there must be only one move involved, and it matches the one symbol of x_i against Y_i , which are the same. Again, we can conclude $Y_i \xRightarrow{*} x_i$; this time, zero steps are used. Now we have the derivation

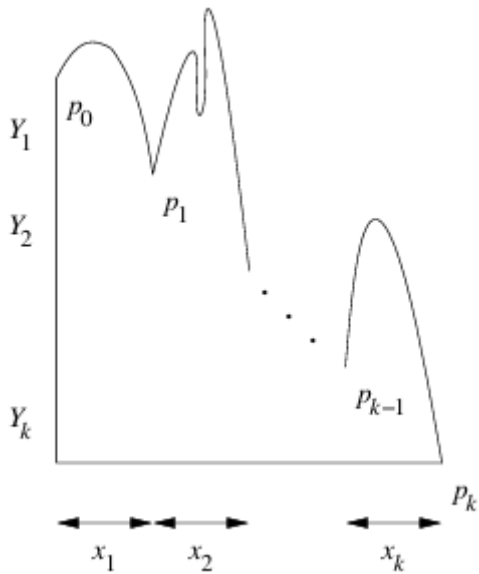
$$A \Rightarrow Y_1 Y_2 \cdots Y_k \xRightarrow{*} x_1 Y_2 \cdots Y_k \xRightarrow{*} \cdots \xRightarrow{*} x_1 x_2 \cdots x_k$$

That is, $A \xRightarrow{*} x$.

To complete the proof, we let $A = S$ and $x = w$. Since we are given that w is in $N(P)$, we know that $(q, w, S) \vdash^* (q, \epsilon, \epsilon)$. By what we have just proved inductively, we have $S \xRightarrow{*} w$; i.e., w is in $L(G)$. $\square$

From PDA to Grammar

- The fundamental event in the history of a PDA's processing of a given input is the net popping of one symbol off the stack, while consuming some input A.
- PDA may change state as it pops stack symbols. Need to track the state that it enters when it finally pops a level off its stack.



- pop is the net effect of possibly many moves.
- The first move may change Y to some other symbol Z. The next move may replace Z by UV later moves have the effect of popping U and then other moves pop V.
- The net effect is that Y has been replaced by nothing. All the input symbols consumed so far constitute x.
- Also consider the net change of states.

From PDA to Grammar

Our construction of an equivalent grammar uses variables each of which represents an “event” consisting of:

1. The net popping of some symbol X from the stack, and
2. A change in state from some p at the beginning to q when X has finally been replaced by ϵ on the stack.

We represent such a variable by the composite symbol $[pXq]$. Remember that

Construction of CFG from a PDA

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ be a PDA We need to construct a CFG $G = (V, \Sigma, R, S)$,

PROOF: We shall construct $G = (V, \Sigma, R, S)$, where the set of variables V consists of:

1. The special symbol S , which is the start symbol, and
2. All symbols of the form $[pXq]$, where p and q are states in Q , and X is a stack symbol, in Γ .

The productions of G are as follows:

- a) For all states p , G has the production $S \rightarrow [q_0Z_0p]$. Recall our intuition that a symbol like $[q_0Z_0p]$ is intended to generate all those strings w that cause P to pop Z_0 from its stack while going from state q_0 to state p . That is, $(q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon)$. If so, then these productions say that start symbol S will generate all strings w that cause P to empty its stack, after starting in its initial ID.

- b) Let $\delta(q, a, X)$ contain the pair $(r, Y_1Y_2 \cdots Y_k)$, where:

1. a is either a symbol in Σ or $a = \epsilon$.
2. k can be any number, including 0, in which case the pair is (r, ϵ) .

Then for all lists of states $r_1, r_2, \dots, r_k$, G has the production

$$[qXr_k] \rightarrow a[rY_1r_1][r_1Y_2r_2] \cdots [r_{k-1}Y_kr_k]$$

This production says that one way to pop X and go from state q to state r_k is to read a (which may be ϵ), then use some input to pop Y_1 off the stack while going from state r to state r_1 , then read some more input that pops Y_2 off the stack and goes from state r_1 to r_2 , and so on.

Theorem 6.14: Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ be a PDA. Then there is a context-free grammar G such that $L(G) = N(P)$.

We need to prove that

- $[qXp] \xRightarrow{*} w$ if and only if $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$.

(If) Suppose $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$. We shall show $[qXp] \xRightarrow{*} w$ by induction on the number of moves made by the PDA.

BASIS: One step. Then (p, ϵ) must be in $\delta(q, w, X)$, and w is either a single symbol or ϵ . By the construction of G , $[qXp] \rightarrow w$ is a production, so $[qXp] \Rightarrow w$.

INDUCTION: Suppose the sequence $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$ takes n steps, and $n > 1$. The first move must look like

$$(q, w, X) \vdash (r_0, x, Y_1 Y_2 \cdots Y_k) \vdash^* (p, \epsilon, \epsilon)$$

where $w = ax$ for some a that is either ϵ or a symbol in Σ . It follows that the pair $(r_0, Y_1 Y_2 \cdots Y_k)$ must be in $\delta(q, a, X)$. Further, by the construction of G , there is a production $[qXr_k] \rightarrow a[r_0 Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{k-1} Y_k r_k]$, where:

1. $r_k = p$, and
2. $r_1, r_2, \dots, r_{k-1}$ are any states in Q .

In particular, we may observe, as was suggested in Fig. 6.10, that each of the symbols $Y_1, Y_2, \dots, Y_k$ gets popped off the stack in turn, and we may choose r_i to be the state of the PDA when Y_i is popped, for $i = 1, 2, \dots, k - 1$. Let $x = w_1 w_2 \cdots w_k$, where w_i is the input consumed while Y_i is popped off the stack. Then we know that $(r_{i-1}, w_i, Y_i) \vdash^* (r_i, \epsilon, \epsilon)$.

As none of these sequences of moves can take as many as n moves, the inductive hypothesis applies to them. We conclude that $[r_{i-1} Y_i r_i] \xRightarrow{*} w_i$. We may put these derivations together with the first production used to conclude:

$$\begin{aligned} [q X r_k] &\Rightarrow a[r_0 Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{k-1} Y_k r_k] \xRightarrow{*} \\ &aw_1[r_1 Y_2 r_2][r_2 Y_3 r_3] \cdots [r_{k-1} Y_k r_k] \xRightarrow{*} \\ &aw_1 w_2[r_2 Y_3 r_3] \cdots [r_{k-1} Y_k r_k] \xRightarrow{*} \\ &\cdots \\ &aw_1 w_2 \cdots w_k = w \end{aligned}$$

where $r_k = p$.

(Only-if) The proof is an induction on the number of steps in the derivation.

BASIS: One step. Then $[q X p] \rightarrow w$ must be a production. The only way for this production to exist is if there is a transition of P in which X is popped and state q becomes state p . That is, (p, ϵ) must be in $\delta(q, a, X)$, and $a = w$. But then $(q, w, X) \vdash (p, \epsilon, \epsilon)$.

INDUCTION: Suppose $[q X p] \xRightarrow{*} w$ by n steps, where $n > 1$. Consider the first sentential form explicitly, which must look like

$$[q X r_k] \Rightarrow a[r_0 Y_1 r_1][r_1 Y_2 r_2] \cdots [r_{k-1} Y_k r_k] \xRightarrow{*} w$$

where $r_k = p$. This production must come from the fact that $(r_0, Y_1 Y_2 \cdots Y_k)$ is in $\delta(q, a, X)$.

We can break w into $w = aw_1 w_2 \cdots w_k$ such that $[r_{i-1} Y_i r_i] \xRightarrow{*} w_i$ for all $i = 1, 2, \dots, k$. By the inductive hypothesis, we know that for all i ,

$$(r_{i-1}, w_i, Y_i) \vdash^* (r_i, \epsilon, \epsilon)$$

If we use Theorem 6.5 to put the correct strings beyond w_i on the input and below Y_i on the stack, we also know that

$$(r_{i-1}, w_i w_{i+1} \cdots w_k, Y_i Y_{i+1} \cdots Y_k) \vdash^* (r_i, w_{i+1} \cdots w_k, Y_{i+1} \cdots Y_k)$$

If we put all these sequences together, we see that

$$\begin{aligned} (q, aw_1 w_2 \cdots w_k, X) &\vdash (r_0, w_1 w_2 \cdots w_k, Y_1 Y_2 \cdots Y_k) \vdash^* \\ (r_1, w_2 w_3 \cdots w_k, Y_2 Y_3 \cdots Y_k) &\vdash^* (r_2, w_3 \cdots w_k, Y_3 \cdots Y_k) \vdash^* \cdots \vdash^* (r_k, \epsilon, \epsilon) \end{aligned}$$

Since $r_k = p$, we have shown that $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$.

We complete the proof as follows. $S \xRightarrow{*} w$ if and only if $[q_0 Z_0 p] \xRightarrow{*} w$ for some p , because of the way the rules for start symbol S are constructed. We just proved that $[q_0 Z_0 p] \xRightarrow{*} w$ if and only if $(q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon)$, i.e., if and only if P accepts x by empty stack. Thus, $L(G) = N(P)$. $\square$

Example 6.15 : Let us convert the PDA $P_N = (\{q\}, \{i, e\}, \{Z\}, \delta_N, q, Z)$ from Example 6.10 to a grammar. Recall that P_N accepts all strings that violate, for the first time, the rule that every e (else) must correspond to some preceding i (if). Since P_N has only one state and one stack symbol, the construction is particularly simple. There are only two variables in the grammar G :

- a) S , the start symbol, which is in every grammar constructed by the method of Theorem 6.14, and
- b) $[qZq]$, the only triple that can be assembled from the states and stack symbols of P_N .

The productions of grammar G are as follows:

1. The only production for S is $S \rightarrow [qZq]$. However, if there were n states of the PDA, then there would be n productions of this type, since the last state could be any of the n states. The first state would have to be the start state, and the stack symbol would have to be the start symbol, as in our production above.

2. From the fact that $\delta_N(q, i, Z)$ contains (q, ZZ) , we get the production $[qZq] \rightarrow i[qZq][qZq]$. Again, for this simple example, there is only one

3. From the fact that $\delta_N(q, e, Z)$ contains (q, ϵ) , we have production

$$[qZq] \rightarrow e$$

Notice that in this case, the list of stack symbols by which Z is replaced is empty, so the only symbol in the body is the input symbol that caused the move.

We may, for convenience, replace the triple $[qZq]$ by some less complex symbol, say A . If we do, then the complete grammar consists of the productions:

$$\begin{aligned} S &\rightarrow A \\ A &\rightarrow iAA \mid e \end{aligned}$$

In fact, if we notice that A and S derive exactly the same strings, we may identify them as one, and write the complete grammar as

$$G = (\{S\}, \{i, e\}, \{S \rightarrow iSS \mid e\}, S)$$

- Section 6.3 of Chapter 6